Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity.

College Name: Vidyasagar college of arts and science

College Code: BRU30

Team ID: SWTID-2026-6624

Team Members:

Team Leader Name: Gomathi G

Email: ggomathi3101@gmail.com

Team Member: Dhinesh S

Email: Crazydinesh179@gmail.com

Team Member: Gowtham Raj V

Email: gowthamgkgowtham796@gmail.com

Team Member: Dharmeshwaran A

Email: dharmeshwarankrishnan@gmail.com

Team Member: Sri Hariharan S

Email: sri04562196@gmail.com

PROJECT DETAILS

Title: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

• AI-Enhanced User Protection (AIUP)

AI-Enhanced User Protection (AIUP) is a next-generation security framework that leverages artificial intelligence and machine learning to verify user identities and protect online platforms from cyber threats. Unlike traditional security measures that rely on static credentials like passwords or even two-factor authentication (2FA), AIUP continuously analyzes user behavior to create a dynamic, evolving "trust score" for each individual. AIUP is essential for modern online security because it addresses the limitations of perimeter-based defenses. As cyber threats become more sophisticated—with attackers using stolen credentials, phishing, and social engineering—relying solely on "what you know" (passwords) or "what you have" (tokens) is no longer sufficient. AIUP adds a third, continuous layer: "who you are" based on how you behave. This system helps organizations prevent account takeovers, reduce fraud, and ensure that the person interacting with a platform is genuinely who they claim to be, throughout their entire session.

Abstract:

Zero Trust Network Access (ZTNA) is an advanced cybersecurity framework designed to secure modern digital environments by eliminating implicit trust and continuously verifying users and devices before granting access to applications. Traditional network security models rely on perimeter-based defenses, which are increasingly ineffective in today's cloud-driven and remote work environments. ZTNA addresses these challenges by adopting the principle of **"never trust, always verify."** This project focuses on implementing **Zero Trust Network Access using Zscaler**, a leading cloud security platform that provides secure and seamless connectivity between users and applications without exposing the internal network. Zscaler enables identity-based access control, continuous authentication, and device posture verification to ensure that only authorized users can access specific resources. The study explores how Zscaler's ZTNA architecture protects enterprise applications by replacing traditional VPNs with a cloud-based security model that provides secure user-to-application connectivity. It demonstrates how policies, authentication mechanisms, and secure access gateways are used to enforce granular access control and minimize the risk of cyber threats such as unauthorized access, data breaches, and lateral movement within networks. Through this implementation, the project highlights the benefits of ZTNA, including enhanced security, improved scalability, reduced attack surface, and better user experience for remote and hybrid work environments. The findings emphasize the importance of adopting Zero Trust principles in modern cybersecurity strategies to protect sensitive data and enterprise infrastructure.

Use Case Scenario:

1. Secure Remote Employee Access

Scenario:
A remote employee tries to access a company's internal HR application from their personal laptop using the internet.

ZTNA Process (Zscaler):

1. User logs into the company portal.
2. Zscaler verifies the user's **identity through authentication (SSO / MFA)**.
3. Device posture check is performed (OS, antivirus status).
4. Zscaler policy engine checks whether the user has permission to access the HR application.
5. Secure connection is established directly to the application.

Output:

- If verified → **Access granted only to HR application.**
- If verification fails → **Access denied.**

Result:
The employee can securely access the application without exposing the internal network.

2. Unauthorized User Access Attempt

Scenario:
An attacker tries to access the company's internal finance application using stolen credentials.

ZTNA Process (Zscaler):

1. User attempts login.
2. Zscaler performs **multi-factor authentication (MFA)**.

3.The system detects unusual login behavior (new location/device).

4.Zscaler security policy blocks the connection.

Output:

- **Access denied.**
- Security alert generated.

Result:
Unauthorized users cannot access sensitive applications even if credentials are compromised.

3. Access from Unsecured Device

Scenario:
A legitimate employee attempts to access a company application from a device without updated antivirus software.

ZTNA Process (Zscaler):

1. User authentication is successful.
2. Zscaler performs a **device posture check**.
3. Device fails security compliance requirements.
4. Zscaler policy restricts access.

Output:

- **Access blocked or limited access granted.**

Result:
Only secure and compliant devices can access company resources.

4. Secure Access to Cloud Applications

Scenario:
An employee needs to access a cloud-based application such as internal project management software.

ZTNA Process (Zscaler):

1. User connects through the Zscaler cloud gateway.

2. Identity verification occurs.
3. Zscaler establishes a **secure encrypted connection** to the cloud application.
4. Access control policies are applied.

Output:

- **User securely accesses the cloud application.**

Result:
Sensitive data remains protected while allowing secure cloud access.

5. Preventing Lateral Movement in the Network

Scenario:
A compromised device attempts to move laterally within the network to access multiple internal applications.

ZTNA Process (Zscaler):

1. Zscaler detects unusual traffic patterns.
2. Access is limited to **only the authorized application**.
3. Attempt to access other internal resources is blocked.

Output:

- **Access to unauthorized applications denied.**
- Security event logged.

Result:
Attackers cannot move across the network because ZTNA restricts access at the application level.

STAGE-1

TITLE: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity.

OVERVIEW:

The project focuses on developing an AI-based identity verification system designed to strengthen online security by analysing and validating user identities through their behavioural patterns on digital platforms. The system utilises advanced machine learning technologies to build detailed user profiles that help differentiate between legitimate users and potentially suspicious activities. It aims to address major cybersecurity concerns such as unauthorised access, fraudulent transactions, and other cyber threats affecting various online services and platforms.

The main objective of this system is to improve digital security by continuously analysing and confirming user identities based on their online behaviour. To achieve this, the system incorporates advanced machine learning methods including deep learning, neural networks, and natural language processing (NLP). These technologies help create comprehensive behavioural profiles for each user, which continuously update and adapt as the user's behaviour changes over time. A key feature of the system is real-time anomaly detection, which identifies unusual activities that deviate from established behaviour patterns and responds quickly to minimise potential security risks.

Additionally, the system integrates multi-factor authentication (MFA) to further enhance security by triggering additional verification steps whenever suspicious behaviour is detected. Notification mechanisms are included to alert both users and administrators, ensuring clear communication and providing guidance for resolving potential security concerns. Secure data storage and management mechanisms are also implemented to protect sensitive user profiles and anomaly detection data.

The system includes an integration layer that allows seamless interaction with different online platforms and services. Monitoring and reporting tools are incorporated to track system performance and security activities, ensuring transparency and accountability. To support increasing numbers of users, the system is designed with scalability and load-balancing capabilities. Strong security practices are implemented by encrypting data during transmission and storage, deploying firewalls and intrusion detection systems, and complying with relevant data protection and privacy regulations.

Cloud infrastructure may also be utilised to provide greater flexibility, scalability, and efficient resource management. Continuous training and optimisation of machine learning models help the system adapt to evolving user behaviour and emerging cyber threats. Furthermore, deploying system components across multiple geographic locations can improve reliability, redundancy, and overall system performance.

List of Vulnerability Table:

S.NO	VULNERABILITY NAME	CWE - NO
1	Broken Access Control	CWE-23 Relative Path Traversal
2	Security Misconfiguration	CWE -16: Configuration
3	Software Supply Chain Failures	CWE-447 use of obsolete Function
4	Cryptographic Failures	CWE –331 Insufficient Entropy
5	Injection	CWE- 20 Improper Input Validation
6	Insecure Design	CWE- 501 Trust Boundary Violation
7	Authentication Failures	CWE- 384 Session Fixation
8	Software or Data Integrity Failures	CWE-426 Untrusted Search Path

9	Security Logging and Alerting Failures	CWE- 778 Insufficient Logging
10	Mishandling of Exceptional Conditions	CWE – 369 Divide By Zero

Report:

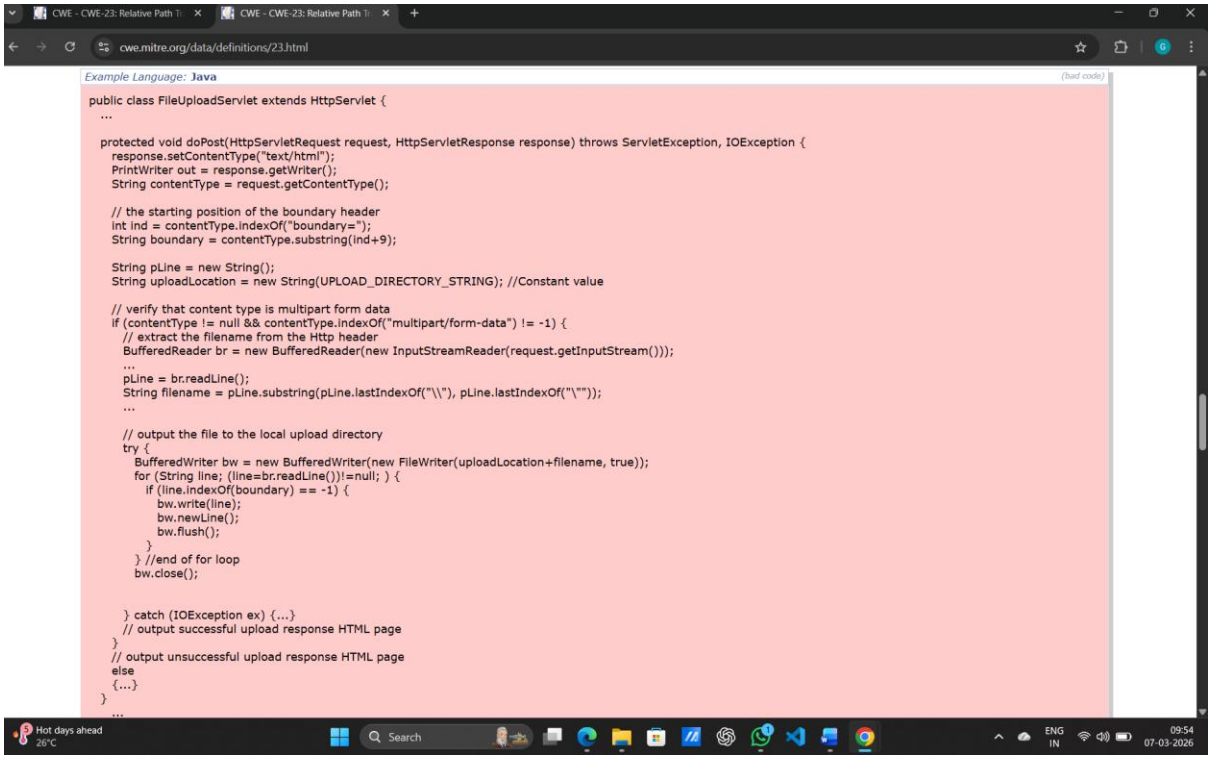
1. **Vulnerability Name:** Broken Access Control

   CWE – 23: Relative Path Traversal

   OWASP Category: A01:2025-Broken Access Control

   Description: The product uses external input to construct a path name that should be within a restricted directory, but it does not properly neutralize sequences such as ".." that can resolve to a location that is outside of that directory.

   Business impact: The attacker may be able read the contents of unexpected files and expose sensitive data by traversing the file system to access files or directories that are outside of the restricted directory. If the targeted file is used for a security mechanism, then the attacker may be able to bypass that mechanism.



Recommandations:

- **Avoid Direct User Input:** The most effective way to prevent path traversal is to avoid passing user-supplied input directly to filesystem APIs.
- **Use Indirect References:** Instead of accepting a filename, use an index number or a unique ID that maps to a specific file in a server-side lookup table.
- **Implement "Allow Lists":** Only accept input that matches a strict list of permitted values or characters (e.g., alphanumeric only).
- **Apply the Principle of Least Privilege:** Ensure the application runs with the minimum necessary permissions so it cannot access sensitive system files even if a traversal occurs.

2. **Vulnerability Name:** Security Misconfiguration

   CWE-16: Configration

   OWASP Category: A01:2025- Security Misconfiguration
   Description: It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services. Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges). Default accounts and their passwords are still enabled and unchanged. A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
   Business Impact: Insecure settings can cause application crashes or allow attackers to trigger Denial of Service (DoS), preventing customers from reaching services and resulting in lost revenue.

Vulnerable Example (Node.js/Express):

javascript

```javascript
app.use((err, req, res, next) => {
  // Directly sending the error object reveals stack traces to the user
  res.status(500).send(err.stack);
});
```

Use code with caution.

Secure Configuration:

javascript

```javascript
app.use((err, req, res, next) => {
  console.error(err.stack); // Log internally for developers
  res.status(500).send("An internal error occurred. Please try again later.");
});
```

Use code with caution.

Ask anything

Recommadations:

- **Automate Hardening:** Develop a repeatable hardening process to deploy environments that are appropriately locked down. Use automation to ensure development, QA, and production environments are configured identically.
- **Maintain a Minimal Platform:** Remove or do not install unused features, components, frameworks, documentation, and samples to reduce the attack surface.
- **Enforce Least Privilege:** Implement strict access controls and grant users only the minimum permissions necessary for their roles.

3. **Vulnerability Name:** Software Supply Chain Failures

   CWE-447: Use of Obsolete Function

   CWE-1021: A03:2025 - Software Supply Chain Failures

Description: A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.

Business Impact: CWE-447 occurs when a user interface displays features that are not actually implemented or supported by the system. This can confuse users and lead to frustration when they **try** to use non-functional options. As a result, user trust in the application may decrease, and organizations may receive more support requests from users reporting these issues. Over time, this can reduce productivity, increase maintenance efforts, and negatively affect the company's reputation.

HTML

```html
<!DOCTYPE html>
<html>
<head>
<title>Profile Page</title>
<script>
function deleteAccount() {
    alert("Feature coming soon!");  // Not implemented
}
</script>
</head>
<body>

<h2>User Profile</h2>
<button onclick="deleteAccount()">Delete Account</button>

</body>
</html>
```

Recommendation:
Applications should avoid displaying features in the user interface that are not fully implemented or supported. Developers should

either remove such features from the UI or clearly disable them until they are completely developed and tested. Proper validation should also be implemented to ensure that unsupported functions cannot be triggered from the frontend or backend. Regular code reviews and UI testing should be conducted to identify incomplete features before deployment. This helps improve user experience, maintain application reliability, and prevent potential security risks.

4. **Vulnerability Name:** Cryptographic Failures

   CWE-331: Insufficient Entropy

   OWASP Category: A04:2025- Cryptographic Failures

   Description: The product uses an algorithm or scheme that produces insufficient entropy, leaving patterns or clusters of values that are more likely to occur than others.
   Business Impact: An attacker could guess the random numbers generated and could gain unauthorized access to a system if the random numbers are used for authentication and authorization.

```csharp
// VULNERABLE: Standard Random() is not cryptographically secure
public string GenerateOTP(int length) {
    string characters = "0123456789";
    string otp = "";
    Random rand = new Random(); // Statistical PRNG
    for (int i = 0; i < length; i++) {
        int index = rand.Next(0, characters.Length);
        otp += characters[index];
    }
    return otp;
}
```

Recommandations:

- **Eliminate Predictable Seeding:**

- Never seed an RNG with easily guessable data such as the system clock, process IDs, or hardware serial numbers.Modern CSPRNG APIs typically handle seeding automatically using OS-level entropy; avoid manual seeding unless using a high-entropy source.

- **Increase Output Length (Bit Size):**
  - Ensure the generated value has enough bits of entropy to prevent brute-force attacks.
- **Avoid Hard-Coded Secret Values:**
  - Do not use hard-coded salts, initialization vectors (IVs), or keys.Ensure these values are unique and randomly generated for every operation.

5. **Vulnerability Name:** Injection

   CWE- 114: Process Control

   OWASP Category: A05:2025- Injection

   Description: Process control vulnerabilities of the first type occur when either datum enters the application from an untrusted source and the data is used as part of a string representing a command that is executed by the application. By executing the command, the application gives an attacker a privilege or capability that the attacker would not otherwise have.
   Business Impact: Exploitation can lead to unauthorized code execution that crashes services or halts production lines, resulting in significant revenue loss and missed delivery deadlines.

Example Language: **C**

```
...
RegQueryValueEx(hkey, "APPHOME",
0, 0, (BYTE*)home, &size);
char* lib=(char*)malloc(strlen(home)+strlen(INITLIB));
if (lib) {

  strcpy(lib,home);
  strcat(lib,INITCMD);
  LoadLibrary(lib);
}
...
```

Recommandations:

Specify Absolute Paths: Always use the absolute path when loading libraries or executing external programs.

Bad: System.loadLibrary("helper.dll"); (Searches system paths which an attacker might influence).

Good: System.load("C:\\App\\libs\\helper.dll"); (Directly targets the intended file).

Use Hardcoded Paths: Whenever possible, hardcode the paths to required libraries and executables within the application to prevent them from being influenced by external configuration files or registry keys.

6. **Vulnerability Name:** Insecure Design

CWE- 501: Trust Boundary Violation

OWASP Category: A06:2025- Insecure Design

Description: A trust boundary can be thought of as line drawn through a program. On one side of the line, data is untrusted. On the other side of the line, data is assumed to be trustworthy. The purpose of validation logic is to allow data to safely cross the trust boundary - to move from untrusted to trusted. A trust boundary violation occurs when a program blurs the line between what is trusted and what is untrusted. By combining trusted and

untrusted data in the same data structure, it becomes easier for programmers to mistakenly trust unvalidated data.

Business Impact:

- **Unauthorized Code Execution**: Attackers can exploit this to execute arbitrary commands with the privileges of the current user, potentially leading to a complete system compromise.
- **Loss of Confidentiality**: Sensitive information, such as administrator passwords or private encryption keys, can be fully divulged to an attacker.

Example Language: **Java**

```
usrname = request.getParameter("usrname");
if (session.getAttribute(ATTR_USR) == null) {
  session.setAttribute(ATTR_USR, usrname);
}
```

Example Language: **C#**

```
usrname = request.Item("usrname");
if (session.Item(ATTR_USR) == null) {
  session.Add(ATTR_USR, usrname);
}
```

Recommandations:

- **Establish Clear Trust Boundaries**: Explicitly define which parts of your application and data structures are "trusted" versus "untrusted".
- **Perform Strict Input Validation**: Always validate data before storing it in a session or using it in a trusted context.
- **Allow-list Validation**: Compare input against a list of known-good values or patterns (e.g., regex) and reject everything else.

- **Plausibility Checks**: Integrate checks at every tier (frontend to backend) to ensure the data makes sense for its intended use.

7. **Vulnerability Name:** Authentication Failures

   CWE- 384 :Session Fixation

   OWASP Category: A07:2025 - Authentication Failures

   Description: Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier gives an attacker the opportunity to steal authenticated sessions.

   Business Impact: Session Fixation occurs when an attacker forces a user to use a known session ID, allowing the attacker to hijack the session after the user logs in. This can lead to unauthorized access to user accounts and sensitive information. As a result, organizations may experience data breaches, financial loss, and misuse of user accounts. It can also damage the company's reputation and reduce customer trust in the system.

   Example Language: **HTML**

```html
<form method="POST" action="j_security_check">
  <input type="text" name="j_username">
  <input type="text" name="j_password">
</form>
```

   Recommendation:

   To prevent session fixation attacks, applications should generate a new session ID after a user successfully logs in and invalidate any existing session IDs. Secure session management practices such as using strong random session IDs, HTTPS encryption, and proper session timeout should be implemented. Developers should also ensure that session IDs are not exposed in URLs or stored insecurely.

Regular security testing and monitoring can help detect and prevent session-related vulnerabilities.

8. **Vulnerability Name:** Software or Data Integrity Failures

CWE- 426: Untrusted Search Path

OWASP Category: A08:2025- Software or Data Integrity Failures

Description: This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Business Impact: The program could be redirected to the wrong files, potentially triggering a crash or hang when the targeted file is too large or does not have the expected format.

Example Language: **C** *(bad co*

```
#define DIR "/restricted/directory"

char cmd[500];
sprintf(cmd, "ls -l %480s", DIR);
/* Raise privileges to those needed for accessing DIR. */

RaisePrivileges(...);
system(cmd);
DropPrivileges(...);
...
```

Recommandations:

- **Sanitize Environment Variables:** Before calling external commands, clear or hard-code the PATH, LD_LIBRARY_PATH, and other search-related variables to known-safe values.
- **Avoid Dangerous Search Functions**: In C/C++, use functions like execl() or execv() which require explicit paths, rather than system(), which relies on the shell's untrusted search path.
- **Secure Service Executables (Windows**): Ensure all service paths are enclosed in double quotes if they contain spaces to prevent the "Unquoted Service Path" variant of this vulnerability.

9. **Vulnerability Name:** Security Logging and Alerting Failures

   CWE- 778: Insufficient Logging

   OWASP Category: A09:2025 - Security Logging and Alerting Failures

   Description: When a security-critical event occurs, the product either does not record the event or omits important details about the event when logging it.

   Business Impact: Insufficient logging occurs when a system does not properly record important events such as user activities, errors, or security incidents. This makes it difficult for organizations to detect and investigate security attacks or system failures. Without proper logs, identifying the source of a problem or unauthorized access becomes challenging. This can lead to data breaches, operational disruptions, financial losses, and damage to the organization's reputation.

Example Language: **C**

```
const int DivideByZero = 10;
double divide(double x, double y){
  if ( 0 == y ){
    throw DivideByZero;
  }
  return x/y;
}
...
try{
  divide(10, 0);
}
catch( int i ){
  if(i==DivideByZero) {
    cerr<<"Divide by zero error";
  }
}
```

   Recommendation: Organizations should implement proper logging mechanisms to record important system events such as user logins, access attempts, errors, and security-related activities. Logs should be detailed, securely stored, and regularly monitored to detect suspicious behavior. Access to logs should be restricted to authorized personnel, and logs should be protected from

modification or deletion. Regular audits and automated monitoring tools should also be used to quickly identify and respond to security incidents.

10. **Vulnerability Name:** Mishandling of Exceptional Conditions

CWE – 369: Divide By Zero

OWASP Category: A10:2025- Mishandling of Exceptional Conditions

Descriptions: This weakness typically occurs when an unexpected value is provided to the product, or if an error occurs that is not properly detected. It frequently occurs in calculations involving physical dimensions such as size, length, width, and height.

Business Impact: This weakness typically occurs when an unexpected value is provided to the product, or if an error occurs that is not properly detected. It frequently occurs in calculations involving physical dimensions such as size, length, width, and height.

Example Language: **C**

```
const int DivideByZero = 10;
double divide(double x, double y){
  if ( 0 == y ){
    throw DivideByZero;
  }
  return x/y;
}
...
try{
  divide(10, 0);
}
catch( int i ){
  if(i==DivideByZero) {
    cerr<<"Divide by zero error";
  }
}
```

Recommandations:

- **Input Sanitization**: For user-provided data, use "allow-lists" or range checks to ensure the value falls within a safe, non-zero interval.

- **Check Collection Sizes:** When calculating averages or percentages based on the count of items in a list, verify the collection is not empty.

STAGE-2

TITLE: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity.

Overview:

Nessus is a vulnerability scanning tool created by Tenable, Inc.. It functions as a remote scanner, which means it can analyse systems without needing to be installed directly on the target machine. The tool works by sending multiple types of probes to the target system to detect potential security weaknesses. These probes may include network packets, operating system commands, and requests sent to applications. Nessus operates using a plugin-based framework. Plugins are small programs designed to detect particular security vulnerabilities. More than 1,200 plugins are available in Nessus, enabling it to identify a wide range of issues across operating systems, software applications, and network devices. When a scan begins, Nessus first uses discovery plugins to determine the operating system and the services running on the target host. After identifying this information, it executes multiple plugins that test the system for possible vulnerabilities. For every vulnerability detected, Nessus generates a detailed report that includes the severity level, instructions to reproduce the issue, and recommended steps to fix or mitigate the problem. Nessus can scan different types of targets, such as individual computers, entire networks, and web applications. It also allows users to scan specific ports and services when required. Scans can be performed manually or scheduled to run automatically at specific times.

Working Process of Nessus:

The following steps explain how Nessus performs a vulnerability scan:

1. Nessus establishes a connection with the target system using protocols such as TCP, UDP, and ICMP.
2. It sends multiple probes to identify the operating system and active services running on the host.
3. The scanner then executes various plugins to check for known vulnerabilities.

4. After completing the scan, Nessus generates a report that outlines the detected vulnerabilities, their severity levels, reproduction steps, and possible remediation methods.

Nessus is widely used by cybersecurity professionals to detect and fix vulnerabilities within networks. Its ease of use and detailed reporting features help organizations prioritize and address security issues effectively.

Advantages of Using Nessus:

- Wide vulnerability detection: With more than 1,200 plugins, Nessus can identify numerous vulnerabilities across operating systems, applications, and network devices.
- User-friendly interface: The tool provides an intuitive interface that simplifies the configuration and execution of scans.
- Detailed reporting: Nessus produces comprehensive reports describing vulnerability severity, reproduction procedures, and remediation steps.
- Scalability: It can scan various targets including individual hosts, full networks, and web applications.
- Flexible scanning: Users can run scans manually or schedule them to run automatically.

Overall, Nessus is a powerful security scanning solution that helps organizations strengthen their network security. It serves as an important tool for cybersecurity professionals aiming to identify and resolve system vulnerabilities.

Target Website: http://testaspnet.vulnweb.com

Target IP address: 44.228.249.3

List of Vulnerability Name:

S.NO	Vulnerability Name	Severity	Plugins
1.	SQL Injection	Critical	11219
2.	Cross-Site Scripting (XSS)	High	10815

3.	Cross-Site Request Forgery (CSRF)	Medium	33850
4.	Local File Inclusion (LFI)	High	59066
5.	Information Disclosure	Low	10863
6.	Security Misconfiguration	Medium	42873
7.	Cookie Not Marked HttpOnly	Low	22964
8.	Detailed Error Messages (Database Errors)	Medium	46180

Report:

1.Vulnerability Name: SQL Injection

Severity: Critical
Plugin ID: 11219
Port: 80 / TCP

Description:
The web application does not properly validate user input. Attackers can inject malicious SQL statements through input fields to access or manipulate backend database data.

Solution:

- Use **parameterized queries / prepared statements**.
- Validate and sanitize user inputs.
- Implement Web Application Firewall (WAF).

Business Impact:
Attackers may access sensitive information such as customer records, login credentials, or financial data, leading to **data breaches and regulatory penalties**.

2. Vulnerability Name: Cross-Site Scripting (XSS) (CWE-79)

Severity: High
Plugin ID: 10815
Port: 80 / TCP

Description:
User input is not properly encoded before being displayed in web pages. Attackers can inject malicious scripts that execute in victims' browsers.

Solution:

- Implement **output encoding**.
- Validate user input.
- Deploy **Content Security Policy (CSP)**.

Business Impact:
Attackers may steal session cookies, redirect users to malicious websites, or perform unauthorized actions.

3. Vulnerability Name: Cross-Site Request Forgery (CSRF) (CWE-352)

Severity: Medium
Plugin ID: 33850
Port: 80 / TCP

Description:
The application lacks proper CSRF protection mechanisms, allowing attackers to trick authenticated users into performing unintended actions.

Solution:

- Implement **CSRF tokens**.
- Use **SameSite cookies**.
- Validate HTTP request origin.

Business Impact:
Unauthorized actions may be performed by users unknowingly, leading to compromised accounts or altered data.

4.Vulnerabilitty Name: Local File Inclusion (LFI) (CWE-98)

Severity: High
Plugin ID: 59066
Port: 80 / TCP

Description:
Improper input validation allows attackers to include local files from the server, potentially exposing sensitive information.

Solution:

- Validate file path parameters.
- Restrict access using file whitelisting.
- Disable unnecessary file inclusion functions.

Business Impact:
Attackers may access confidential system files and gain insights into the system configuration.

5.Vulnerability Name: Information Disclosure (CWE-200)

Severity: Low
Plugin ID: 10863
Port: 80 / TCP

Description:
The web server reveals sensitive information such as server type or software versions through HTTP headers.

Solution:

- Remove **Server** and **X-Powered-By** headers.
- Configure secure server settings.

Business Impact:
Attackers can identify software versions and exploit known vulnerabilities.

6.Vulnerability Name: Security Misconfiguration (CWE-16**)**

Severity: Medium
Plugin ID: 42873
Port: 80 / TCP

Description:
The application contains insecure configuration settings such as missing security headers or default configurations.

Solution:

- Implement security headers.
- Disable unnecessary services.
- Follow security hardening guidelines.

Business Impact:
Misconfiguration may allow attackers to exploit weaknesses in the application environment.

7. Vulnerability Name: Cookie Not Marked HttpOnly (CWE-1004)

Severity: Low
Plugin ID: 22964
Port: 80 / TCP

Description:
Session cookies are not configured with the HttpOnly flag, allowing them to be accessed through client-side scripts.

Solution:

- Enable **HtpOnly** and **Secure** flags for cookies.
- Implement secure session management.

Business Impact:
Attackers may steal session cookies through XSS and hijack user sessions.

8. Vulnerability Name: Detailed Error Messages (CWE-209)

Severity: Medium
Plugin ID: 46180
Port: 80 / TCP

Description:
The application reveals detailed error messages that expose database queries or system information.

Solution:

- Disable detailed error messages in production.
- Implement generic user error responses.
- Log detailed errors internally.

Business Impact:
Attackers can gather system information that helps them plan more advanced attacks.

STAGE 3

TITLE: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity.

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a modern cybersecurity model that enables secure remote access to an organization's internal applications and data through strict access control rules. Unlike traditional Virtual Private Networks (VPNs), which often allow users to access the entire network once connected, ZTNA follows the principle of "never trust, always verify." This means users are granted access only to the specific applications they require, and only after their identity and device security status are validated. ZTNA plays an essential role in modern security infrastructures. It helps organizations move beyond the traditional "castle-and-moat" security model, where internal networks were considered safe while external networks were treated as untrusted. Instead, ZTNA hides internal resources from the public internet and creates direct, encrypted connections between users and applications. This significantly reduces the attack surface and prevents threats such as attackers moving laterally within a network after gaining initial access.

The ZTNA Access Cycle

The ZTNA access cycle is an ongoing process that verifies user identity, establishes secure communication channels, and enforces access policies whenever a request is made. The typical stages include:

1. User Request

A user from any location—such as an office, home, or public place—requests access to a particular internal application, for example an HR system, financial server, or internal development repository.

2. Identity and Device Authentication

Before granting access, the ZTNA system verifies the user's identity using methods such as multi-factor authentication (MFA). It also evaluates the security condition of the device being used, ensuring that antivirus software is updated, the firewall is active, and the device has not been compromised.

3. Policy Validation

The system then checks predefined access policies to determine whether the specific user, using a particular device, is allowed to access the requested application at that time.

4. Secure Connection Creation

If the request meets policy requirements, the ZTNA system establishes a secure and encrypted micro-tunnel between the user's device and the application. Importantly, the user gains access only to that application rather than the entire network.

5. Continuous Monitoring

Even after access is granted, the system continuously observes the session for unusual activity. If suspicious behavior or device changes are detected, the connection can be immediately terminated.

6. Logging and Analysis

All access attempts—both successful and unsuccessful—are recorded and analyzed. These logs support security audits, threat detection, and improvements in access policies.

Zscaler (Zscaler Internet Access / Zscaler Private Access)

Zscaler provides a cloud-native security platform that delivers its services entirely through the cloud, removing the need for traditional on-premises hardware such as VPN gateways.

For implementing ZTNA, Zscaler offers **Zscaler Private Access (ZPA)**. This service enables smooth, identity-based access to private applications without placing users directly on the organization's internal network.

The ZPA architecture works on a simple idea: the application is effectively placed within the Zscaler cloud environment, and users connect to it securely through that platform. To enable secure access, the system gathers information from several sources, including:

1. Identity Providers (IdP)

Services such as **Microsoft Azure Active Directory**, **Okta**, or **Ping Identity** are used to authenticate and verify user identities.

2. Device Security Platforms

Security tools like **CrowdStrike Falcon**, **Tanium**, or **Microsoft Defender for Endpoint** help determine the health and security status of the user's device.

3. Business Applications

These include internal web applications, file servers, and legacy systems hosted either in private data centers or public cloud environments such as **Amazon Web Services** or **Microsoft Azure**.

4. Zscaler App Connectors

App Connectors are lightweight software components installed within the organization's data center or cloud infrastructure. They create outbound connections to the Zscaler cloud, ensuring that internal servers remain hidden from the internet and are not directly exposed to external threats.

Policy Enforcement in Zscaler

Zscaler enforces security policies by analyzing several factors such as user identity, device condition, and application access. The main policy mechanisms include:

1. **Identity-Based Access** – Access rules are defined using user identities and group memberships rather than relying on IP addresses.
2. **Application Segmentation** – Users are allowed to access only the specific applications they require instead of entire network segments.

3. **Least Privilege Principle** – Users are granted only the minimum permissions necessary to perform their tasks, reducing the risk of misuse or compromise.

The Zscaler ZTNA Cycle

The ZTNA cycle implemented through Zscaler applies Zero Trust principles in a practical environment using cloud infrastructure. The typical workflow includes the following stages:

1. User Authentication

When a user tries to access an internal application, they are redirected to the organization's Identity Provider for authentication. Multi-Factor Authentication (MFA) is also applied to verify the user.

2. Device Security Validation

Zscaler integrates with endpoint security solutions to confirm that the user's device meets organizational security requirements before granting access.

3. Policy Evaluation

The Zscaler cloud platform analyzes the access request and checks it against predefined security policies. It verifies whether the user, device, time, and location meet the criteria to access the requested application.

4. Creation of Secure Micro-Tunnel

Once the request is approved, Zscaler establishes an encrypted micro-tunnel between the user's device and the target application through the Zscaler cloud.

5. Traffic Inspection (Optional)

For approved applications, Zscaler can optionally inspect traffic to detect malware or enforce data loss prevention (DLP) policies.

6. Activity Logging and Monitoring

All activities are recorded within the Zscaler cloud portal, enabling security teams to perform analysis, reporting, and compliance verification.

MISP Integration (Malware Information Sharing Platform)

While Zscaler primarily manages secure application access, its capabilities can be strengthened by integrating threat intelligence platforms such as **Malware Information Sharing Platform**.

1. Threat Intelligence Integration

MISP collects and distributes threat intelligence related to malicious IP addresses, domains, and URLs. These indicators of compromise (IOCs) can be integrated into Zscaler's cloud platform.

2. Proactive Threat Blocking

If an authorized user attempts to connect to a malicious website identified through the MISP feed, **Zscaler Internet Access** can block the connection, preventing potential security incidents.

3. Improved Threat Visibility

Security analysts can correlate Zscaler activity logs with MISP intelligence data. If unusual behavior is detected, analysts can determine whether the destination domain or IP is linked to known cyber-attack campaigns.

College Network Infrastructure

The college network environment contains several components that increase its exposure to cyber threats, particularly with the expansion of remote learning and administrative access. The infrastructure includes:

- Core routers and switches
- Distribution routers and switches
- Wireless access points
- Firewalls and Intrusion Detection Systems (IDS)
- Internal application servers hosting student systems, library services, research repositories, and administrative portals
- A legacy VPN concentrator used by faculty and staff for remote connectivity

However, relying on traditional VPN technology introduces several limitations.

Limitations of the Current VPN Model

1. **Excessive Network Access**
   Once connected through VPN, a compromised device could potentially access and attack multiple internal systems.
2. **Limited External Collaboration**
   Some research platforms must be accessed by external partners, but VPN access exposes too much of the network.
3. **Poor User Experience**
   Legacy VPN systems can be slow and inefficient because all user traffic must pass through the university network.

Deploying ZTNA with Zscaler in the College Network

To implement ZTNA using **Zscaler Private Access**, the focus should shift from protecting the network to controlling access to applications.

The deployment plan may include the following steps:

1. Application Identification

List all internal applications such as ERP systems, library databases, research servers, and file-sharing platforms. Determine which applications currently rely on VPN access.

2. User Segmentation

Divide users into groups such as:

- Students
- Research faculty
- Administrative staff
- IT administrators
- External collaborators

Each group should only access applications relevant to their role.

3. Deployment of App Connectors

Install Zscaler App Connector virtual machines within the campus data center. These connectors establish secure outbound connections to the Zscaler cloud while keeping internal servers hidden from the internet.

4. Identity Provider Integration

Integrate Zscaler with the university's existing identity system, such as **Microsoft Azure Active Directory**, allowing authentication through existing credentials and MFA policies.

5. Access Policy Configuration

Define detailed access policies. For example:
Faculty members may access the ERP system only after MFA authentication and only if their device meets security requirements.

6. Client Connector Deployment

Students, faculty, and staff install the Zscaler Client Connector on their devices. This client securely routes traffic to private applications through ZPA and internet traffic through ZIA.

Threat Intelligence in the ZTNA Environment

Threat intelligence improves security decisions by providing contextual insights into access requests.

Key Sources

1. Zscaler cloud activity logs
2. External threat intelligence platforms such as MISP
3. Identity provider authentication logs

Applications of Threat Intelligence

- Adaptive access control based on contextual risk
- Detection of compromised user accounts
- Blocking communication with malicious domains and servers

Incident Response in a ZTNA Model

Incident response becomes faster and more efficient in a Zero Trust architecture because access is tightly controlled.

1. Detection

Security teams identify suspicious behavior, such as abnormal data downloads from a server.

2. Containment

Administrators can immediately revoke the user's access through the Zscaler management portal, preventing further connections.

3. Eradication

The compromised device is isolated and examined for malware by IT staff.

4. Recovery

Once the issue is resolved and credentials are reset, access can be safely restored.

5. Post-Incident Review

Security teams analyze logs to determine the cause of the breach and update policies to prevent similar incidents.

Benefits

- Rapid containment through cloud controls
- Prevention of lateral movement within the network
- Detailed forensic logs for investigation
- Improved regulatory compliance

Understanding the Zscaler ZPA Tool

Zscaler Private Access is a cloud-based solution designed to connect users directly to applications instead of exposing the entire network.

Key features include:

1. **Application-focused security** – Policies are created based on applications rather than network segments.

2. **Hidden infrastructure** – Internal servers remain invisible to the internet because App Connectors only create outbound connections.
3. **Dark cloud architecture** – Applications remain hidden until authentication and verification are completed.
4. **Improved user experience** – The Zscaler Client Connector operates silently without the complexity of traditional VPNs.
5. **High scalability** – Being cloud-based, the system can support thousands of users without requiring additional hardware.

Conclusion

This project explored the role of artificial intelligence in web security using **IBM QRadar** across several stages.

The first stage introduced web application testing and highlighted the importance of identifying vulnerabilities to maintain secure online systems. This foundational knowledge supported the later stages of the project.

The second stage focused on analyzing vulnerability reports generated by **Nessus**, which helped identify potential security weaknesses and guided the implementation of effective remediation strategies.

The third stage examined the concept of Zero Trust Network Access and its implementation through Zscaler Private Access. This approach demonstrated a transition from traditional VPN-based access models to identity-based, application-level connectivity. By hiding internal resources and enforcing strict access policies, ZTNA significantly reduces the risk of cyberattacks and unauthorized access.

Overall, the project emphasized the importance of adopting modern security approaches to address the challenges posed by remote work environments and cloud technologies.

Topics Explored:

Data Collection and Aggregation, Standardization and Normalisation, Enrichment and Analysis, Sharing and Distribution, Integration with

Security Tools, Customization and Flexibility, Automation and Orchestration, Community Involvement, Incident Response Support, Threat Intelligence Feeds, Soc and Siem

Tools explored:

Nmap, Metasploit, Nessus, Kali Linux, OWASP ZAP, Burp Suite